

Backend Handoff: Consolidated Report API

To: Ward (Backend Developer)
From: Selym (Design/Scoping Agent)
Date: 2026-04-09
Deadline: Production ready by 2026-04-17

Executive Summary

You're building the API endpoint that powers the DevOps HQ consolidated report feature. The frontend will display task status, blockers, owner, and risk level in a dashboard with a "Refresh" button.

Your deliverables: 1. Database schema (blockers table, indexes)
2. API endpoint: `POST /api/v1/dashboard/consolidated-report` 3. Data aggregation queries + business logic 4. Caching layer (Redis) 5. Tests + load benchmarking 6. Documentation

Timeline: 5 business days (parallel with frontend dev)
Start: Friday 2026-04-11
End: Thursday 2026-04-17

API Contract

Endpoint Specification

`POST /api/v1/dashboard/consolidated-report`
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
Timeout: 10 seconds

REQUEST BODY:

```
{
  "filters": {
    "team": "platform" | null,
    "status": "blocked" | "in-progress" | "proposed" | "completed"
  } | null,
```

```
    "owner": "ward@example.com" | null,  
    "riskLevel": "high" | "medium" | "low" | null,  
    "limit": 50,  
    "offset": 0  
  }  
}
```

RESPONSE (200 OK):

```
{  
  "summary": {  
    "total": 25,  
    "completed": 12,  
    "inProgress": 5,  
    "proposed": 3,  
    "blocked": 2,  
    "atRiskCount": 3,  
    "velocity": 2.4  
  },  
  
  "blockers": [  
    {  
      "id": "blocker-uuid",  
      "taskId": "task-uuid",  
      "taskTitle": "API Schema Design",  
      "reason": "Backend team hasn't delivered schema",  
      "owner": "ward@example.com",  
      "ownerName": "Ward",  
      "riskLevel": "high",  
      "ageHours": 72,  
      "suggestedAction": "escalate"  
    }  
  ],  
  
  "tasks": [  
    {  
      "id": "task-uuid",  
      "title": "API Schema Design",  
      "status": "blocked",  
      "owner": "ward@example.com",  
      "ownerName": "Ward",  
      "riskLevel": "high",  
      "percentComplete": 0,  
      "updatedAt": "2026-04-06T14:32:00Z",  
      "blockerCount": 1  
    }  
  ],  
  
  "generatedAt": "2026-04-09T19:00:00Z",  
  "cacheHit": false,  
  "executionTimeMs": 1240  
}
```

ERROR RESPONSES:

```
{
  "error": "INVALID_FILTER",
  "message": "Invalid status filter",
  "code": "E400_BAD_REQUEST"
}
```

Full contract: See ARCHITECTURE_RECOMMENDATION.md (pages on API Contract section).

Database Schema

Tables to Create/Modify

-- BLOCKERS table (NEW)

```
CREATE TABLE blockers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  reason TEXT NOT NULL,
  owner_id UUID REFERENCES auth.users(id),
  risk_level TEXT DEFAULT 'medium', -- 'low' | 'medium' | 'high'
  created_at TIMESTAMP DEFAULT now(),
  resolved_at TIMESTAMP NULL,
  CONSTRAINT blockers_resolved_check CHECK (resolved_at IS NULL
    OR resolved_at > created_at)
);

CREATE INDEX idx_blockers_task_id ON blockers(task_id);
CREATE INDEX idx_blockers_owner_id ON blockers(owner_id);
CREATE INDEX idx_blockers_created_at ON blockers(created_at DESC);
```

-- Enable RLS

```
ALTER TABLE blockers ENABLE ROW LEVEL SECURITY;
```

-- RLS Policy: Users see blockers for tasks in their team

```
CREATE POLICY "Users see blockers for their tasks"
ON blockers
FOR SELECT
USING (
  task_id IN (
    SELECT id FROM tasks WHERE team_id IN (
      SELECT team_id FROM team_members WHERE user_id = auth.uid()
    )
  )
);
```

-- TASKS table modifications

-- Assumption: tasks table already exists

-- Verify it has these columns:

```

-- id UUID
-- title TEXT
-- status TEXT ('proposed' | 'in-progress' | 'completed' |
--             'blocked')
-- assigned_to UUID (FK to auth.users)
-- risk_level TEXT ('low' | 'medium' | 'high')
-- percent_complete INT (0-100)
-- team_id UUID (for multi-team isolation)
-- updated_at TIMESTAMP
-- created_at TIMESTAMP

-- Add indexes if not already present
CREATE INDEX IF NOT EXISTS idx_tasks_status ON tasks(status);
CREATE INDEX IF NOT EXISTS idx_tasks_assigned_to ON
    tasks(assigned_to);
CREATE INDEX IF NOT EXISTS idx_tasks_risk_level ON
    tasks(risk_level);
CREATE INDEX IF NOT EXISTS idx_tasks_team_id ON tasks(team_id);

-- TASK OWNERS VIEW (convenient for queries)
CREATE OR REPLACE VIEW task_owners_view AS
SELECT
    t.id,
    t.title,
    t.status,
    t.team_id,
    t.risk_level,
    t.percent_complete,
    t.updated_at,
    u.id as owner_id,
    u.email as owner_email,
    (u.user_metadata->>'full_name') as owner_name,
    COUNT(CASE WHEN b.resolved_at IS NULL THEN 1 END) as
        open_blocker_count,
    MAX(CASE WHEN b.resolved_at IS NULL THEN b.created_at END) as
        oldest_blocker_date
FROM tasks t
LEFT JOIN auth.users u ON t.assigned_to = u.id
LEFT JOIN blockers b ON t.id = b.task_id
WHERE t.deleted_at IS NULL
GROUP BY t.id, u.id, u.email;

```

Migration file: Create /migrations/001_add_blockers_table.sql

Data Aggregation Service

Functions to Implement

1. aggregateTasksByStatus(teamId, offset?, limit?)

Returns: { proposed, inProgress, completed, blocked }

```
type TaskStatusSummary = {
  proposed: number;
  inProgress: number;
  completed: number;
  blocked: number;
};

async function aggregateTasksByStatus(
  teamId: string,
  offset = 0,
  limit = 50
): Promise<{ summary: TaskStatusSummary; total: number }> {
  // SQL query:
  // SELECT status, COUNT(*) FROM tasks
  // WHERE team_id = $1 AND deleted_at IS NULL
  // GROUP BY status
}
```

2. getOpenBlockers(teamId, sortBy = 'risk')

Returns: Array of blockers sorted by risk + age

```
interface Blocker {
  id: string;
  taskId: string;
  taskTitle: string;
  reason: string;
  owner: string;
  ownerName: string;
  riskLevel: 'high' | 'medium' | 'low';
  ageHours: number;
  suggestedAction: 'escalate' | 'unblock' | 'ping';
}

async function getOpenBlockers(
  teamId: string,
  limit = 20
): Promise<Blocker[]> {
  // SELECT blockers with task info
  // WHERE team_id = $1 AND resolved_at IS NULL
  // ORDER BY risk_level DESC, created_at ASC
  // Suggest action based on age:
}
```

```

    // - ageHours > 72 → 'escalate'
    // - ageHours > 24 → 'ping'
    // - else → 'unblock'
}

```

3. calculateVelocity(teamId, weeksPast = 4)

Returns: Tasks completed per week (rolling average)

```

async function calculateVelocity(
  teamId: string,
  weeksPast = 4
): Promise<number> {
  // SELECT COUNT(*) FROM tasks
  // WHERE status = 'completed'
  // AND updated_at > now() - (weeksPast * '1 week'::interval)
  // THEN divide by weeksPast
}

```

4. getTaskSnapshot(teamId, filters, offset, limit)

Returns: Array of tasks with owner info

```

interface TaskSnapshot {
  id: string;
  title: string;
  status: string;
  owner: string;
  ownerName: string;
  riskLevel: string;
  percentComplete: number;
  updatedAt: string; // ISO8601
  blockerCount: number;
}

```

```

async function getTaskSnapshot(
  teamId: string,
  filters: {
    status?: string;
    owner?: string;
    riskLevel?: string;
  },
  offset = 0,
  limit = 50
): Promise<TaskSnapshot[]> {
  // SELECT from task_owners_view
  // WHERE team_id = $1 AND (status = $2 if provided) AND ...
  // ORDER BY updated_at DESC
  // LIMIT + OFFSET
}

```

5. consolidatedReport(userId, filters)

Returns: Complete report object

```
interface ConsolidatedReport {
  summary: {
    total: number;
    completed: number;
    inProgress: number;
    proposed: number;
    blocked: number;
    atRiskCount: number;
    velocity: number;
  };
  blockers: Blocker[];
  tasks: TaskSnapshot[];
  generatedAt: string;
  executionTimeMs: number;
}

async function consolidatedReport(
  userId: string,
  filters: {
    team?: string;
    status?: string;
    owner?: string;
    riskLevel?: string;
    limit?: number;
    offset?: number;
  }
): Promise<ConsolidatedReport> {
  const startTime = Date.now();

  // 1. Get user's team(s)
  // 2. Call aggregateTasksByStatus()
  // 3. Call getOpenBlockers()
  // 4. Call calculateVelocity()
  // 5. Call getTaskSnapshot() with filters
  // 6. Count at-risk tasks (HIGH or MEDIUM)
  // 7. Combine into response
  // 8. Return with executionTimeMs
}
```

Critical Questions (Answer by EOD Friday 4/11)

These questions determine how we calculate/display risk and blockers.

Q1: Risk Level Calculation

Current state: Is `risk_level` a field on tasks, or do we calculate it?

Options: - A) Manual field: Team manually sets `risk_level` on each task - B) Auto-calculated: $\text{Risk} = \text{age of task} + \text{priority level}$ - C) Hybrid: Manual field, but auto-increase if overdue

What we need for MVP: Clear formula so frontend can color-code (🟢 low | 🟡 medium | 🔴 high)

Recommendation: Option A (manual field) for MVP. Easy to implement, controllable. Phase 2 can add auto-calculation.

Q2: Blocker Detection

Current state: How do we know a task is blocked?

Options: - A) Status field: `status = 'blocked'` is set manually - B) Auto-detect: Task is blocked if it has unresolved blockers in blockers table - C) Both: `status = 'blocked'` manually set; auto-blocked if has unresolved blockers

What we need for MVP: Clear definition so we surface the right tasks

Recommendation: Option C (both). Set `status = 'blocked'` manually, but also query blockers table. Frontend can show blocker count + reason.

Q3: Single vs. Multiple Owners

Current state: Can a task have multiple owners, or just one?

Options: - A) Single owner: `assigned_to = one user UUID` - B) Multiple owners: junction table `task_owners (task_id, user_id)`

What we need for MVP: Know which field to query + how to display in report

Recommendation: Option A (single owner) for MVP. Easier to implement. Phase 2 can add co-ownership if needed.

Q4: Team Scoping

Current state: Should the report show all tasks, or filtered by team?

Options: - A) Single-team view: Report shows only tasks in user's primary team - B) Multi-team view: User sees all tasks across all their teams - C) Org-wide view: Admins see all tasks organization-wide

What we need for MVP: Know which team_id to filter by

Recommendation: Option A (single-team) for MVP. User's team is determined by their team membership. Phase 2 can add multi-team selector.

Q5: Real-Time Updates

Current state: Do we have Supabase Realtime subscriptions enabled?

Options: - A) Polling: Frontend polls API every 5 min (simple, less real-time) - B) WebSocket: Supabase Realtime subscriptions (real-time, more complex) - C) Hybrid: Poll by default, WebSocket for phase 2

What we need for MVP: Know if we can use Supabase Realtime or stick to polling

Recommendation: Option C (hybrid). MVP uses polling (React Query refetch interval). Phase 2 adds WebSocket if needed.

Q6: Max Task Count

Current state: How many tasks might we need to aggregate?

Ballpark: 50? 500? 5000?

What we need for MVP: Know if we need pagination + how to optimize queries

Recommendation: If <500 per team, fetch all. If >500, paginate (limit 50, offset 0). Test with realistic data volume.

Q7: Existing Caching Infrastructure

Current state: Is Redis already deployed, or do we set it up new?

Options: - A) Existing Redis: We have a Redis instance running, connection string available - B) New Redis: We need to spin up a Redis container/service - C) Alternative cache: Use Supabase cache, or in-memory cache

What we need for MVP: Connection string + any auth needed

Recommendation: Use Redis if available (fast, scalable). If not, start with React Query client-side cache only. Redis can be phase 1.5.

Work Breakdown (Your Timeline)

Day 1 (Fri 4/11): Database + Aggregation

☐

Create blockers table + indexes

☐

Write + test aggregation queries

☐

Set up RLS policies

☐

Sync with frontend: Confirm data structure

Day 2 (Mon 4/14): API Endpoint + Caching

☐

Build API endpoint handler

☐

Integrate Redis caching

☐

Request validation (Zod)

☐

Demo endpoint via curl/Postman

Day 3 (Tue 4/15): Testing + Docs

☐

Unit tests (aggregation functions)

☐

Integration tests (API endpoint)

☐

Load testing (measure P95 latency)

☐

API documentation (Swagger)

Day 4 (Wed 4/16): Polish + Staging

- ☐ Performance tuning (if latency >5s)
- ☐ Deploy to staging environment
- ☐ Verify with frontend team
- ☐ Final integration fixes

Day 5 (Thu 4/17): Production Release

- ☐ Production deployment
- ☐ Monitor error rates + latency
- ☐ Verify real data + RLS enforcement

Deliverables Checklist

Code: - ☐ /migrations/001_add_blockers_table.sql - ☐ /src/services/reportService.ts (aggregation logic) - ☐ /src/routes/dashboard.ts (API endpoint) - ☐ /src/middleware/cacheMiddleware.ts (Redis caching)

Tests: - ☐ /tests/services/reportService.test.ts - ☐ /tests/routes/dashboard.test.ts - ☐ Load test results (P95 latency baseline)

Documentation: - ☐ API endpoint swagger spec - ☐ Database schema documentation - ☐ Setup instructions (Redis, env vars) - ☐ Deployment + rollback procedure

Performance: - ☐ Latency <5s for 100 tasks - ☐ Cache hit rate 80%+ - ☐ Load test: 100 concurrent requests

Communication Plan

Sync meetings: - Daily standup: 10 AM EST (async Slack updates OK) - Integration checkpoint: Wed 4/15 3 PM EST - Go/No-go: Thu 4/17 9 AM EST

Questions/blockers: - Slack: #devops-hq channel - Urgent: @mention in Slack

Demo: - Day 2 end: Curl demo of API endpoint - Day 3 end: Postman collection + docs - Day 4 end: Staging environment walkthrough

Success Criteria

- ☐ API responds in <5s (cold), <1s (cached)
 - ☐ All required fields in response (status, blockers, owner, risk)
 - ☐ RLS enforced (users only see their team's data)
 - ☐ Error handling (400, 401, 500 responses)
 - ☐ Caching working (80%+ hit rate on second refresh)
 - ☐ Tests passing (unit + integration)
 - ☐ Load test: P95 latency <8s at 100 req/s
 - ☐ Documentation complete (Swagger + setup guide)
 - ☐ Staging deployment verified
 - ☐ Production deployment successful
-

Questions?

Reach out to Selym (this agent) with any clarifications needed. The frontend team + Ward should align on the API contract by end of Day 1.

Good luck! 🍀